

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2103

Course Title: Data Structures Lab

Assignment No: 04

Date of Submission: 4/01/2026

Submitted by,

Name: Durjoy Roy

StudentID:00724105101055

Lab Section: B1

Academic Semester: Spring 2025

Program: BSc in CSE

Question:

Implement the below functions of singly linked list in a single .cpp file.

(1) insert first

- (2) insert last
- (3) insert anywhere between two nodes (by position, by value)
- (4) delete first
- (5) delete last
- (6) delete from anywhere between two nodes (by position, by value)
- (7) printing() -> function to print the linked list
- (8) searching() -> function to search a value in the linked list
- (9) last_node() -> function to print the value of the last node
- (10) previous_of_last_node() -> function to print the value of the previous node of last node
- (11) list_size() -> function to print the size of the linked list
- (12) reversePrint() -> function to print the linked list in reverse order
- (13) main function -> no manual node creation/connection is allowed

Solution:

Code :

```
#include<bits/stdc++.h>

using namespace std;

struct node
{
    int data;
```

```
    node* next;  
};
```

```
node* head = NULL;
```

```
void insertFirst(int val)  
{  
    node *temp = new node();  
    temp->data = val;  
    temp->next = head;  
    head = temp;  
}
```

```
void insertLast(int val)  
{  
    node *temp = new node();  
    temp->data = val;  
  
    if(head == NULL)  
    {  
        head = temp;
```

```
        return;
    }

    node *tail = head;
    while(tail->next != NULL)
    {
        tail = tail->next;
    }
    tail->next = temp;
    temp->next = NULL;
}
```

```
void insertMiddle(int pos, int val)
{
    if(pos == 1)
    {
        insertFirst(val);
        return;
    }
    node *temp = new node();
    temp->data = val;
```

```
node *tail = head;
for(int i=1; i<pos-1 && tail != NULL; i++)
{
    tail = tail->next;
}
temp->next = tail->next;
tail->next = temp;
}
```

```
void insertAfterVal(int tar, int val)
{
    node *temp = new node();
    temp->data = val;
    node *curr = head;
    while(curr!=NULL && curr->data!=tar)
    {
        curr = curr->next;
    }
    temp->next = curr->next;
    curr->next = temp;
}
```

```
void deleteFirst()
{
    if(head == NULL) return;
    head = head->next;
}
```

```
void deleteLast()
{
    if(head == NULL) return;
    if(head->next == NULL)
    {
        head = NULL;
        return;
    }
    node *curr = head;
    node *pre = head;
    while(curr->next != NULL)
    {
        pre = curr;
        curr = curr->next;
    }
    pre->next = NULL;
```

```
}
```

```
void deleteMiddle(int pos)
```

```
{
```

```
    if(head == NULL) return;
```

```
    if(pos == 1)
```

```
    {
```

```
        deleteFirst();
```

```
        return;
```

```
    }
```

```
    node *temp = head;
```

```
    node *pre = head;
```

```
    for(int i=1; i<pos && temp->next!=NULL; i++)
```

```
    {
```

```
        pre = temp;
```

```
        temp = temp->next;
```

```
    }
```

```
    pre->next = temp->next;
```

```
    temp->next = NULL;
```

```
}
```

```
void deleteVal(int val)
```

```
{
    if(head == NULL) return;
    if(head->data == val)
    {
        deleteFirst();
        return;
    }
    node *curr = head->next;
    node *pre = head;
    while(curr!=NULL && curr->data != val)
    {
        pre = curr;
        curr = curr->next;
    }
    if(curr==NULL) return;
    pre->next = curr->next;
}
```

void printing()

```
{
    node* curr = head;
    while(curr!=NULL)
```



```
{  
    cout << curr->data << " ";  
    curr = curr->next;  
}  
cout << endl;  
}
```

```
void searching(int val)  
{  
    node *valu = head;  
    int f = 0;  
    while(valu != NULL)  
    {  
        if(valu->data == val)  
        {  
            cout << "Found" << endl;  
            f = 1;  
            break;  
        }  
        valu = valu->next;  
    }  
    if(f == 0) cout << "Not Found" << endl;  
}
```

```
void last_node()
{
    node *curr = head;
    while(curr->next!=NULL)
    {
        curr = curr->next;
    }
    cout << "Last node: " << curr->data << endl;
}
```

```
void previous_of_last_node()
{
    if(head == NULL || head->next == NULL)
    {
        cout << "Previous of last node does not exist\n";
        return;
    }
    node *curr = head;
    node *pre = head;
    while(curr->next!=NULL)
    {
        pre = curr;
```

```
        curr = curr->next;
    }
    cout << "Previous of last node: " << pre->data << endl;
}
```

```
void list_size()
{
    int cnt = 0;
    node *ele = head;
    while(ele!=NULL)
    {
        cnt++;
        ele = ele->next;
    }
    cout << "List size: " << cnt << endl;
}
```

```
void reversePrint(node* root)
{
    if(root!=NULL)
    {
        reversePrint(root->next);
        cout << root->data << " ";
    }
}
```

```
}  
}
```

```
int main()
```

```
{
```

```
    insertLast(40);
```

```
    insertLast(50);
```

```
    insertLast(60);
```

```
    printing();
```

```
    insertFirst(10);
```

```
    printing();
```

```
    insertLast(70);
```

```
    printing();
```

```
    insertMiddle(2, 20);
```

```
    printing();
```

```
    insertAfterVal(20,30);
```

```
    printing();
```

```
deleteFirst();
```

```
printing();
```

```
deleteLast();
```

```
printing();
```

```
deleteMiddle(2);
```

```
printing();
```

```
deleteVal(50);
```

```
printing();
```

```
searching(20);
```

```
last_node();
```

```
previous_of_last_node();
```

```
list_size();
```

```
cout << "Reverse: ";
```

```
reversePrint(head);
```

```
cout << endl;
```

```
    return 0;  
}
```

OUTPUT :

40 50 60

10 40 50 60

10 40 50 60 70

10 20 40 50 60 70

10 20 30 40 50 60 70

20 30 40 50 60 70

20 30 40 50 60

20 40 50 60

20 40 60

Found

Last node: 60

Previous of last node: 40

List size: 3

Reverse: 60 40 20